



Offloading strategy with PSO for mobile edge computing based on cache mechanism

Wenqi Zhou¹ · Lunyuan Chen² · Shunpu Tang¹ · Lijia Lai¹ · Junjuan Xia¹ · Fasheng Zhou² · Liseng Fan¹

Received: 30 April 2021 / Revised: 10 August 2021 / Accepted: 9 September 2021 / Published online: 2 November 2021
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2021

Abstract

With the development of Internet of Things (IoT) devices and the growth of users' demand for computation and real-time services, artificial intelligence has been applied to reduce the system cost for future network systems. To meet the demand of network services, the paradigm of edge networks is increasingly shifting towards the joint design of computation, communication and caching services. This paper investigates a multi-user cache-enabled mobile edge computing (MEC) network and proposes an intelligent particle swarm optimization (PSO) based offloading strategy with cache mechanism. In each time slot, the server selects one file among multiple ones to pre-store, according to the proposed cache replacement strategy. In the next time slot, the requested files by the users needn't to be computed and offloaded, if these files have been cached in the server. For the files that have not been cached in the server, PSO algorithm is adopted to find an appropriate offloading ratio to implement the partial offloading. Simulation results are finally presented to validate the proposed studies. In particular, we can find that incorporating the proposed cache replacement strategy into the computation offloading can effectively reduce the system latency and energy consumption for the future networks.

Keywords Edge computing · Offloading computation · Cache-enabled · Cache replacement

Abbreviations

MEC Mobile edge computing
BSs Base stations
PSO Particle swarm optimization

UAV Unmanned aerial vehicle
ES Edge server
FIFO First in first out

✉ Fasheng Zhou
zhoufs@gzhu.edu.cn

✉ Liseng Fan
lsfan2019@126.com

Wenqi Zhou
2112006056@e.gzhu.edu.cn

Lunyuan Chen
2112019037@e.gzhu.edu.cn

Shunpu Tang
tangshunpu@e.gzhu.edu.cn

Lijia Lai
2112006122@e.gzhu.edu.cn

Junjuan Xia
xiajunjuan@gzhu.edu.cn

¹ School of Computer Science, Guangzhou University, Guangzhou, China

² School of Electronics and Communication Engineering, Guangzhou University, Guangzhou, China

1 Introduction

In recent years, mobile cloud computing has significantly enhanced large-scale data processing [1–4], where large volumes of data are transmitted to and processed in central cloud servers. However, with the rapidly growing amount of data generated by mobile devices at the edge of the cloud and the lack of spectrum resources, the traditional cloud computing is becoming more and more difficult to support mobile services and applications with large amount of data [5–7]. The data transmission has become the bottleneck of the cloud computing paradigm. To solve this problem, MEC has been proposed for providing mobile devices with a low latency and energy consumption, by deploying computation nodes at the edge of networks [8, 9]. As the key technology of MEC networks and IoT, computational offloading is regarded as a solution for the

devices with limited resources. In the literature such as the works [10, 11], the computation offloading problem in edge computing was studied. Partial computation offloading is one of the main computational models in the existing literature on MEC, which partitions the computational tasks into two parts. One part of the tasks is executed at the mobile devices while the other is offloaded to the edge for execution. The strategy of partial offloading can reduce the system cost by optimizing offloading ratio, power control and bandwidth allocation. A lot of related works have been performed to solve the problem of partial offloading in mobile edge computing, such as [12, 13]. In particular, some intelligent algorithms such as the deep learning [14–16] and federated learning [17–19] based techniques can be applied to the MEC networks, in order to find a fine offloading strategy for the system.

With the explosive number of mobile devices in the edge networks and the development of wireless communication technologies such as 5G and forthcoming 6G, the volume of data requested by users is getting larger and larger, especially when the proportion of video streaming is increasing. In fact, some content requests are highly concentrated, and some popular content will be repeatedly requested and computed. Therefore, wireless caching has been proposed to avoid frequent replications for the same content through caching them at the base stations (BSs) [20], and edge servers allow service providers to cache the tasks at the edge of the cloud. In edge caching, the file providers cache popular or commonly used files from the remote data center and store them at the edge of the network e.g., BSs. When users request files that have been cached, the latency, energy consumption and the capacity bottleneck of backhaul networks can be reduced [21]. However, the storage capacity at mobile edges is not infinite. With a large number of popular files that users request, it is impossible to cache all the files in the edge nodes. Therefore, devising an effective caching strategy plays an important role in cache-enabled networks.

The application of wireless caching into MEC networks has attracted much attention from the researchers, which can help reduce the system cost in terms of the latency and energy consumption. In this research area, service caching for the resource allocation is one of the important problems to be solved in the MEC networks. The resources of the current edge server are limited, making it difficult to accommodate all users computation requests. Such a contradiction between the limited resources and increasing demands causes a challenge on how to effectively allocate resources for service caching. Therefore, finding an effective strategy to achieve a better resource allocation is of vital importance in cache-enabled MEC networks [22].

Considering the limited storage space of the edge server and the resource allocation of computation offloading, this

paper proposes a computation offloading strategy based on a cache mechanism. Combining the proposed cache replacement strategy with the PSO based partial offloading strategy can achieve better resource allocation strategy and reduce the system latency and energy consumption. The main contributions of this work are summarized below.

- A caching mechanism is incorporated into the offloading strategy to reduce the system cost caused by repeated requests for files, repeated calculations and offloading.
- The PSO algorithm is adopted to optimize the offloading strategy, and a better offloading strategy is devised to achieve a better resource allocation.
- A cache replacement strategy based on user recommendation is proposed, which can improve the caching accuracy and effectively reduce the latency and energy consumption in the calculation and offloading.
- Simulated experiments are conducted to demonstrate the superiority of the proposed approach over the conventional cache replacement strategies, by evaluating the performance of the proposed resource allocation strategy.

The rest of this paper is described as follows. Section 2 describes the system model and formulates the problem. Section 3 presents the proposed cache replacement strategy and provides the PSO algorithm. In Sect. 4, some simulation results and discussions are given, and we finally conclude this work in Sect. 5.

2 System model and problem formulation

Figure 1 shows the system model of the cached-enabled MEC networks, where there is one edge server (ES) and M mobile users, denoted by $\mathcal{U} := \{u_m | 1 \leq m \leq M\}$. The ES can operate as a small-scale data center to store data and execute tasks requested by the users. For data storage, the

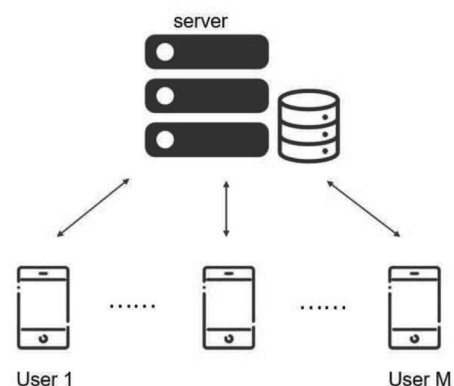


Fig. 1 System model of cache-enabled MEC networks

server can decide which file should be cached according to some factors such as file's popularity and channel condition. When the users request some files for computing, if the file has been cached in the edge server, the server can directly feedback the computational results to the users without offloading and computing. On the contrary, if the file is not stored by the ES, the users have to compute the task assisted by the ES through offloading a part of task. We denote the set of all requested files as $\mathcal{D} := \{d_k | 1 \leq k \leq K\}$, and use the set $\mathcal{X} := \{x_k | 1 \leq k \leq K\}$ to indicate whether the file is cached at the ES or not, where $x_k = 0$ indicates that the file is cached in the server while $x_k = 1$ indicates that the file is not cached in the server. For the files requested by each user, we can denote it as $\mathcal{D}_m := \{d_{m,1}, d_{m,2}, \dots, d_{m,k_m}\}$, where $\mathcal{D}_m \subseteq \mathcal{D}$, and we use $\mathcal{X}_m := \{x_{m,1}, x_{m,2}, \dots, x_{m,k_m}\}$ to indicate whether the file requested by the user is cached or not, where $\mathcal{X}_m \in \mathcal{X}$ and $x_{m,k_m} \in \{0, 1\}$.

2.1 Communication model

We take user u_m as an example to describe the communication and computational models of all users. When user u_m uploads a file through the wireless link, the data rate of the wireless transmission is [23–25]

$$r_m = B_m \log_2 \left(1 + \frac{P_m |h_m|^2}{\sigma^2} \right), \quad (1)$$

where B_m is the bandwidth of the wireless channel, P_m is the transmit power of user m , $h_m \sim \mathcal{CN}(0, 1)$ is the instantaneous channel parameter of the link from user m to the server, and σ^2 denotes the variance of additive white Gaussian noise (AWGN) [26–28].

2.2 Computation model

2.2.1 Local computation model

The tasks can be computed locally or on the MEC server via computation offloading. The corresponding computational latency and energy consumption are mainly relevant to the CPU cycle frequency and the workload of the computational task [29–31]. When some parts of the tasks need to be executed locally, the computation overhead depends on the caching decision, offloading ratio, computational workload of tasks and local CPU cycle frequency. Thus, the local execution latency of user u_m can be written as

$$L_m^l = \frac{(1 - \rho_m) \sum_{i=1}^{k_m} x_{m,i} s_{m,i}}{f_m}, \quad (2)$$

where $\rho_m \in [0, 1]$ is the offloading ratio, $s_{m,i}$ is the data size

of the file $d_{m,i} \in \mathcal{D}_m$, k_m is the number of files requested by user u_m , and f_m is the local CPU cycle frequency of user u_m . The energy consumption of the user u_m can be written as

$$E_m^l = (1 - \rho_m) \left(\sum_{i=1}^{k_m} x_{m,i} s_{m,i} \right) f_m^2 \epsilon_l, \quad (3)$$

where ϵ_l is the energy consumption coefficient of user u_m depending on the local chip architecture [31].

2.2.2 Offloading model

When some parts of the tasks need to be executed by the server, the user should upload the files to the server and the transmission latency and energy consumption of user u_m can be written as

$$L_m^t = \frac{\rho_m \sum_{i=1}^{k_m} x_{m,i} s_{m,i}}{r_m}, \quad (4)$$

$$E_m^t = \frac{\rho_m \sum_{i=1}^{k_m} x_{m,i} s_{m,i}}{r_m} P_m. \quad (5)$$

The computational latency and energy consumption at the server can be written as

$$L_m^{es} = \frac{\rho_m \sum_{i=1}^{k_m} x_{m,i} s_{m,i}}{f_{es}}, \quad (6)$$

$$E_m^{es} = \rho_m \left(\sum_{i=1}^{k_m} x_{m,i} s_{m,i} \right) f_{es}^2 \epsilon_{es}, \quad (7)$$

where f_{es} is the CPU cycle frequency at the edge server and ϵ_{es} is energy consumption coefficient of the server. Since uploading files and doing computation on the server are two operations performed in sequence, we can add these two parts together. Therefore, the offloading latency and energy consumption of user u_m can be written as

$$L_m^o = L_m^t + L_m^{es}, \quad (8)$$

$$E_m^o = E_m^t + E_m^{es}. \quad (9)$$

2.2.3 System cost

For user u_m , the local computation and offloading can be executed in parallel, and thus the latency of user u_m can be written as

$$L_m = \max(L_m^l, L_m^o). \quad (10)$$

The system latency is the maximum value among the latency of all users, and it can be written as

$$L_{total} = \max(L_1, L_2, \dots, L_m). \quad (11)$$

The energy consumption of user u_m can be written as

$$E_m = E_m^l + E_m^o, \quad (12)$$

and the system energy consumption is the summary of the energy consumption of all users, given by

$$E_{total} = \sum_{m=1}^M E_m. \quad (13)$$

2.2.4 Linear combination form of cost

In many practical application scenarios, we need to consider both latency and energy consumption of the system. However, when we need to minimize both the latency and energy consumption of the system, the problem is essentially a multi-objective optimization problem, which is difficult to solve in practice [32]. Motivated by this, we use a linearly weighted form of both latency and energy consumption [33], which can be written as

$$C_{total} = \alpha E_{total} + (1 - \alpha) L_{total}, \quad (14)$$

where $\alpha \in [0, 1]$ is a weight factor. As α becomes larger, the proportion of energy consumption in the system cost increases and the impact of energy consumption plays a more important role in the system. While as α decreases, the proportion of latency factor in the system cost increases and the impact of latency plays a more important role in the system. $\alpha = 1$ means that we only consider the system energy consumption while $\alpha = 0$ means that we only consider the system latency.

2.3 Problem formulation

According to (11), (13) and (14), we can see that the system cost is related to the caching decisions and offloading ratios. We use the set $\mathcal{P} = \{\rho_m | 1 \leq \rho_m \leq M\}$ to represent the offloading decision vector. The optimization target under different application scenarios are also different, so we adopt three commonly used application scenarios and describe the optimization problem in each scenario.

In some energy-limited networks such as Vehicle and UAV networks [7], due to the limited power of mobile devices and the limited computational power of the edge server, energy saving has become an important target of system optimization. For example, UAVs are prone to crash and cause dangerous accidents when their battery is low. In such a scenario, we need to minimize the system energy consumption in order to achieve the goal of energy saving. According to (13), the minimization problem can be expressed as

$$\begin{aligned} \mathbf{P}_1 : \min_{\mathcal{P}, \mathcal{X}_m} E_{total} &= \min_{\mathcal{P}, \mathcal{X}_m} \sum_{m=1}^M E_m \\ \text{s.t. } \rho_m &\in [0, 1], \forall \rho_m \in \mathcal{P}, \\ x_{m,i} &\in \{0, 1\}, \forall x_{m,i} \in \mathcal{X}_m. \end{aligned} \quad (15)$$

In some delay-sensitive scenarios such as decisions based on sensing data, too much delay can lead to a number of problems. For example, when the self-driving car is in the state of automatic driving, long computational latency easily causes traffic accidents because the car can not make a correct judgment in time. Therefore, in such scenarios, reducing the delay becomes the target of the optimization problem. According to (11), the minimization problem of the latency is

$$\begin{aligned} \mathbf{P}_2 : \min_{\mathcal{P}, \mathcal{X}_m} L_{total} &= \min_{\mathcal{P}, \mathcal{X}_m} \max(L_1, L_2, \dots, L_m) \\ \text{s.t. } \rho_m &\in [0, 1], \forall \rho_m \in \mathcal{P}, \\ x_{m,i} &\in \{0, 1\}, \forall x_{m,i} \in \mathcal{X}_m. \end{aligned} \quad (16)$$

For some more general application scenarios, we need to consider both energy saving and latency reduction for the system. With the rapid development of self-driving and new energy vehicles, computational latency and energy consumption of intelligent vehicles are two indicators that we need to consider. A large computational latency makes it impossible for the intelligent vehicles to get the results in time, and excessive energy consumption would accelerate battery consumption. In such scenarios, the latency and energy consumption are both the indicators that need to be optimized. According to (14), we use the linear combination cost to optimize both the energy consumption and latency. The minimization problem of the combination cost is

$$\begin{aligned} \mathbf{P}_3 : \min_{\mathcal{P}, \mathcal{X}_m} C_{total} &= \min_{\mathcal{P}, \mathcal{X}_m} \alpha E_{total} + (1 - \alpha) L_{total} \\ \text{s.t. } \rho_m &\in [0, 1], \forall \rho_m \in \mathcal{P}, \\ x_{m,i} &\in \{0, 1\}, \forall x_{m,i} \in \mathcal{X}_m. \end{aligned} \quad (17)$$

We can observe that the objective function in (15), (16) and (17) are complicated, and the optimization problems are related to the caching strategy and user's offloading decision, where the problems are mixed integer programming (MIP) problems. Therefore, the problems in (15), (16) and (17) are difficult to find the optimal solution and the traditional convex optimization method cannot be used to solve the problems. For MIP problems, some efficient heuristic algorithms can be taken into account. In the following sections, we will propose a caching strategy and adopt the PSO algorithm to optimize the offloading decision in order to minimize the latency, energy consumption and combination cost.

3 Caching and offloading strategy

In this section, the caching strategy is introduced and the caching procedure is given. Then, the PSO algorithm is introduced, and the procedure of optimizing the system cost with PSO is further given.

3.1 Caching strategy

Caching files in the ES can help reduce the system cost of communication and computation, because users can directly get the results from ES without offloading and computing, if the files that the users request has been cached in the ES. However, it is unwise to cache all the files due to the limited caching space and unbearable workload of ES. Therefore, we need to devise a wise caching strategy to decide which files should be cached by ES to reduce the workload and system cost. Inspired by [10], this paper adopts a cache replacement strategy based on users' recommendation, as shown in Fig. 2.

We assume that the cache space of ES is denoted as $\Omega = \{\omega_n | 1 \leq n \leq N\}$, where ω_n is the file cached in the server, and the size of cache space is N . ES can update the cache space to adapt users' demand in each time slot. Caching all request files will increase the workload of ES and users in each time slot. Therefore, only one file should be selected for caching in one time slot according to the popularity of the files. When the cache space is full, the least popular files in the cache space will be replaced by the most popular one. In each time slot, each user selects the

top N popular files and sends the file recommendation table to the ES, and then the ES selects the current N most popular files according to the user's recommendation, which can be denoted as $\mathcal{D}^c = \{d_1^c, \dots, d_N^c\}$. Moreover, according to the cache replacement strategy, if the storage space of the server is not full, the most popular un-cached file $d_n^c \in \mathcal{D}^c$ will be selected for caching. If the storage space is full, the most popular un-cached file d_n^c will be selected to replace the currently unpopular file $\omega_i \notin \mathcal{D}^c$ in the cache space Ω . According to [21], the popularity of each file for user u_m can be expressed as

$$p_m^k = \frac{a_m^k}{a_m}, \quad (18)$$

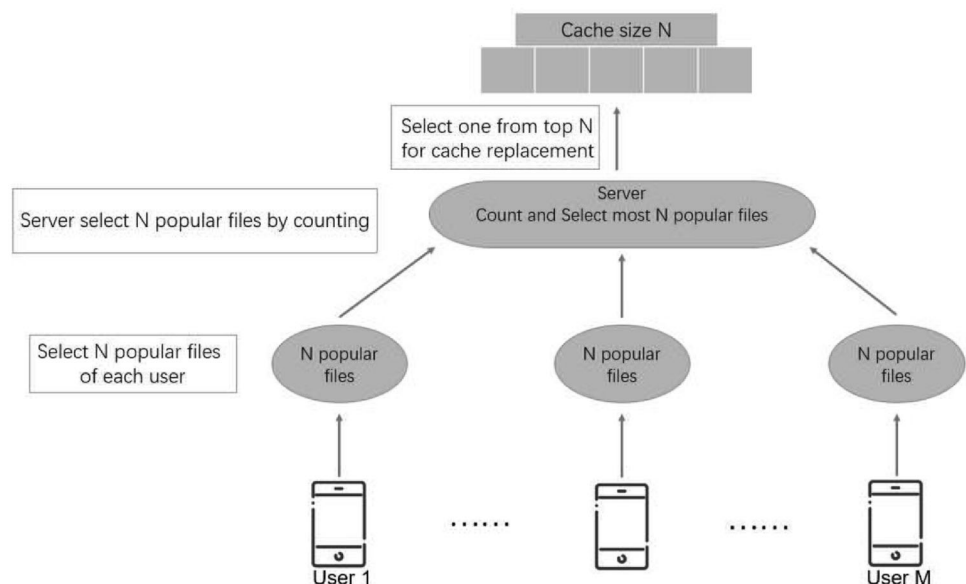
where a_m^k is the requesting times for file d_k of u_m in the current time slot t and a_m is the total requesting times of u_m . Without loss of generality, we assume that the user's preference follows the Zipf distribution which can be expressed as

$$P(r) = \frac{C}{r^\zeta}, \quad (19)$$

where ζ represents the rank of the file, $P(r)$ indicates the occurrence frequency of the file and C is a constant. In [34], Chen and Yang had demonstrated that Zipf distribution was fit for different users with parameter ζ uniformly distributed in $[0.5, 3]$. Hence, we use this model to generate the requested files in the simulation part.

From the above description, we can summarize the procedure of the proposed cache strategy in Algorithm 1.

Fig. 2 Cache replacement strategy based on user's recommendation



Algorithm 1 Cache Replacement Strategy Based on user's recommendation

Input: Top N popular files $\mathcal{D}^c$, cache space Ω , size of the cache space Ω_{size} , the file decided to cache d^* , size of the file β_d

Output: Cache space Ω

```

1: for each time slot  $t=1,2,\dots$  do
2:   for each user parallelly do
3:     calculate  $p_m^k$ 
4:     select  $N$  popular files  $\rightarrow$  server
5:   end for
6:   server count and select top  $N$  files :  $\mathcal{D}^c$ 
7:   if  $\Omega_{size} \geq \beta_d$  then
8:     for in  $\mathcal{D}^c$  do
9:       if  $d_n^c \notin \Omega$  then
10:         $d^* = d_n^c \rightarrow \Omega$ 
11:       end if
12:     end for
13:   else
14:     for in  $\mathcal{D}^c$  do
15:       if  $d_n^c \notin \Omega$  then
16:         $d^* = d_n^c$ 
17:       end if
18:     end for
19:     for in  $\Omega$  do
20:       if  $\omega_n \notin \mathcal{D}^c$  then
21:         $d^*$  replace  $\omega_n$ 
22:       end if
23:     end for
24:   end if
25: end for
  
```

3.2 PSO formulation and offloading strategy

We proceed to optimize the offloading strategy according to the PSO algorithm, which is inspired by the feeding behavior of birds and imitate this behavior. The search space of the optimization problem is analogous to the flight space of birds, and each bird is viewed as a particle to represent a feasible solution of the problem. The optimal solution to the optimization problem is equivalent to the food source that the birds search. PSO is initialized into a group of random particles (random solutions), and the optimal solution is found through T_p iterations, where we denote the number of the total particles as J . In each iteration, the particle updates its velocity v and position x by finding two extreme values $pbest$ and $gbest$, where $pbest$ and $gbest$ are the particle's best position and global best position of the previous iteration, respectively. Each particle updates its velocity toward its $pbest$ and $gbest$, and the velocity in iteration t_p can be written as:

$$v_j^{t_p} = v_j^{t_p-1} + c_1 \times rand(0, 1) \times (pbest_j^{t_p-1} - z_j^{t_p-1}) + c_2 \times rand(0, 1) \times (gbest^{t_p-1} - z_j^{t_p-1}), \quad (20)$$

where c_1 and c_2 are two acceleration constants, which represent the weights of acceleration terms that guide each

particle toward $pbest$ and $gbest$, respectively. When the values of c_1 and c_2 are large, the search speed will be fast and the optimal solution may be missed. When the values of c_1 and c_2 are small, the search speed will be slow and a local optimal value may be caused. Hence, we take $c_1 = 2$ and $c_2 = 2$. Meanwhile, the position of each particle is updated by velocity:

$$z_j^{t_p} = z_j^{t_p-1} + v_j^{t_p}. \quad (21)$$

After T_p iterations, the global best position $gbest$ will be used as the final solution of PSO.

In summary, PSO is adopted to optimize the offloading decision for minimizing the latency, energy consumption and the combination cost. The procedure of the PSO based offloading strategy with the cache mechanism is summarized in Algorithm 2.

Algorithm 2 PSO Based Offloading Strategy With the Cache Mechanism

Input: User's request file $d_{m,k_m} \in \mathcal{D}_m$, cache space Ω , number of PSO population J , PSO iteration T_p , cost calculation function $F(\cdot)$

Output: Minimal cost C^* , offloading decision vector $\mathcal{P}^*$

```

1: for each time slot  $t=1,2,\dots,T$  do
2:   users send request  $d_{m,k_m}$  to server
3:   if  $d_{m,k_m}$  has been cached in  $\Omega$  then
4:      $x_{m,k_m} = 0$ 
5:   end if
6:   offloading with PSO:
7:   initialize  $C^*$ 
8:   randomly initialize  $\mathcal{P}_j, V_j$ 
9:   initialize  $pbest_j, gbest$ 
10:  for each iteration  $t_p = 1, 2, \dots, T_p$  do
11:    for  $j = 1$  to  $J$  do
12:      update  $\mathcal{P}_j$  and  $V_j$ 
13:      if  $F(\mathcal{P}_j) < F(pbest_j)$  then
14:         $pbest_j = \mathcal{P}_j$ 
15:      end if
16:      if  $F(pbest_j) < C^*$  then
17:         $gbest = pbest_j$ 
18:         $C^* = F(pbest_j)$ 
19:      end if
20:    end for
21:  end for
22:   $\mathcal{P}^* = gbest$ 
23:  update cache space by the proposed cache strategy
24: end for
  
```

4 Results and discussions

4.1 Parameter setting

In this part, four caching and offloading strategies are adopted for comparison in the simulation, which are listed as follows,

- Proposed strategy: the proposed strategy jointly employs the PSO method and the cache replacement strategy based on the user's recommendation.
- PSO offloading with random cache (PSO-random cache): this strategy jointly employs the PSO method and the cache replacement strategy with random selection of files. Specifically, a file is selected randomly from the requested files for caching instead of the user's recommendation. When the cache space is full, the selected file will randomly replace the file in the cache space.
- PSO offloading with first in first out (FIFO) replacement (PSO-FIFO cache): this strategy jointly employs the PSO method and the cache replacement strategy of FIFO. Specifically, a file is selected from the requested files for caching based on the user's recommendation. When the cache space is full, the selected file will replace the file in cache space according to FIFO.
- PSO offloading without caching (PSO-without cache): this strategy adopts PSO method to offload without caching files in the server.

For the above four strategies, we assume that the wireless channel and user's preference remain unchanged within one time slot and the requested files are randomly generated for each user according to (19) with $\zeta = 2$ and $C = 1$ in the simulations. Without loss of generality, we set the channel gains of the network to unity, in order to focus on the study of caching and offloading design. In addition, we perform the simulations by setting $B_m = 40$ MHz, $P_m = 2$ W and $\sigma^2 = 1 \times 10^{-9}$ W. Moreover, for the convenience of comparison and analysis, we assume that the number of the files generated by users remains unchanged with the time slots [35–37], and the size of each file is the same, thus we set $s_{m,i} = 80$ MHz. In further, we set the CPU cycle frequency of the user and server to 400 MHz and 800 MHz respectively, and the energy consumption coefficients of the user and server are set to 1×10^{-26} and 1×10^{-28} ,

Table 1 Parameters setting

Parameter	Value
Bandwidth of the wireless channel B_m	40 MHz
Transmit power P_m	2 W
Variance of additive white Gaussian noise σ^2	1×10^{-9} W
Data size of each file $s_{m,i}$	80 Mbit
Local CPU cycle frequency f_m	400 MHz
Server CPU cycle frequency f_{es}	800 MHz
User's energy consumption coefficient ϵ_l	1×10^{-26}
Server's energy consumption coefficient ϵ_{es}	1×10^{-28}

respectively. The parameter setting is summarized in Table 1.

4.2 Simulation results

Figure 3 shows the system energy consumption of the three strategies versus the time slot, where the cache size is set to 5 and the number of users is set to 3. From the results in Fig. 3, we can observe that the energy consumption of the proposed strategy decreases with the time slot and it becomes convergent after six time slots. Moreover, the energy saving effect of the proposed strategy is better than that of the PSO-without and PSO-random cache. This is because that the proposed strategy can effectively exploit the cache resources by caching the files according to the user's recommendation. In contrast, the PSO-random cache decreases in the first six time slots, while it fluctuates from six time slots. This is because that this scheme randomly chooses files to cache and the files with a high popularity would be replaced by the files with a low popularity when random caching replacement occurs, which leads to an increase and fluctuation of the energy consumption in the last five time slots. The energy consumption of the PSO-without cache remains almost unchanged with the time slot, since it cannot make use of the cache resources. From these phenomena, we can find that the proposed strategy can outperform the conventional two strategies by effectively exploiting the caching resources.

Figure 4 depicts the effect of cache size on system energy consumption of three strategies., where the number of users is set to 3. From Fig. 4, the results show that due to the increase of cache space, more files can be cached in advance, so that the energy consumption of the three strategies are reduced. Moreover, the energy consumption

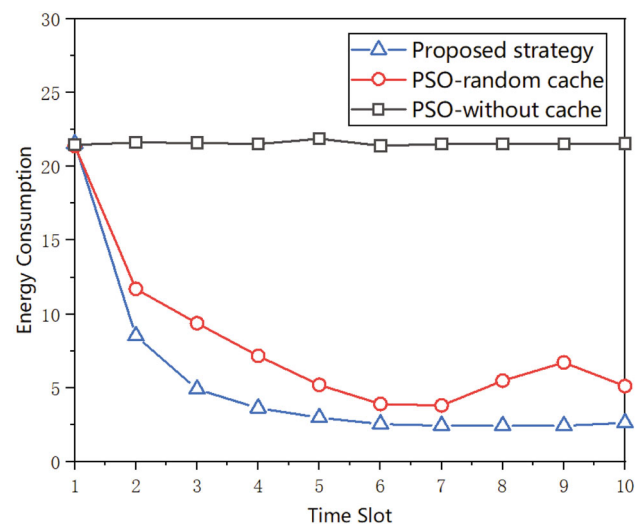


Fig. 3 Energy consumption of three strategies versus time slot

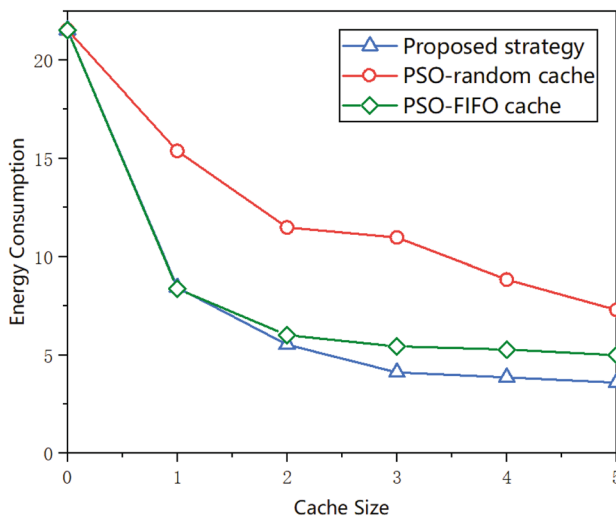


Fig. 4 Effect of cache size on the energy consumption of three strategies

of the proposed strategy is lower than that of PSO-random cache and PSO-FIFO cache when the cache size is larger than 1. This is because that the proposed strategy is based on the cache replacement strategy by user's recommendation, and its accuracy of caching files is better than that of the other two strategies when the cache size is large. When the cache size is less than 2, the cost of proposed scheme and PSO-FIFO scheme almost coincides. The reason is that PSO-FIFO scheme always caches the first or the second popular file when cache size is set to 1, where the effect on the energy consumption can be ignored. From the above phenomena, we can find that the proposed strategy are better than the two conventional strategies in terms of the energy consumption by effectively using the caching resources.

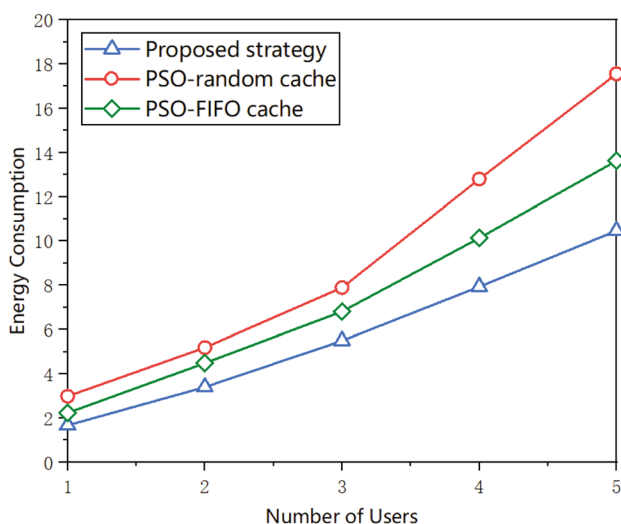


Fig. 5 Impact of the number of users on the energy consumption of three strategies

Figure 5 demonstrates the results of the system energy consumption for the three strategies varies with the number of users, where the cache size is set to 5. From the results in Fig. 5, we can observe that the energy consumption of the three strategies increases with the number of users. This is because when the number of users increases, there are more files to be offloaded and computed, which results in a larger energy consumption. Moreover, the energy consumption of the PSO-random cache and PSO-FIFO cache is higher than that of the proposed strategy, as its caching accuracy is better than that of the other two strategies. In contrast, PSO-random cache and PSO-FIFO cache consume much more energy due to the random and FIFO replacement. From these phenomena, we can find that the proposed strategy is superior to the two conventional strategies in the system energy consumption.

Figure 6 illustrates the system latency of the three strategies versus the time slots, where the cache size is set to 5 and the number of users is set to 3. From the results in Fig. 6, we can observe that the latency of the proposed strategy decreases with the time slot, and it becomes convergent after six time slots. Moreover, the latency of the proposed strategy is smaller than that of the PSO-without and PSO-random cache, since it can prevent the files with a high popularity being replaced by the files with a low popularity, so that the server can choose the top five files with the highest popularity. In contrast, PSO-random cache scheme randomly chooses files to replace the files in the cache space, and it results in the popular files being replaced out of the cache space, leading to an increase and fluctuation of the latency in the last five time slots. In further, the latency of the PSO-without cache remains almost unchanged with the time slot, since it cannot make use of the cache resources.

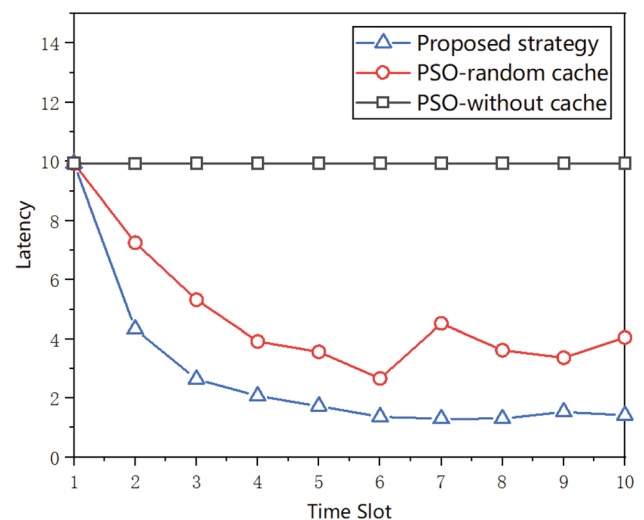


Fig. 6 Latency of three strategies versus time slot

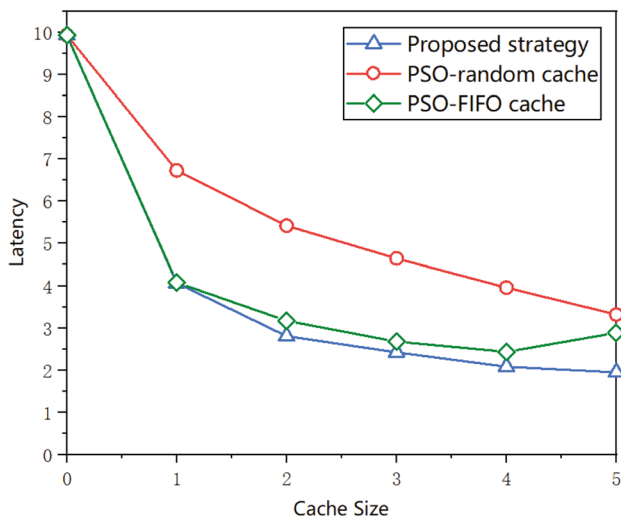


Fig. 7 Effect of cache size on the latency of three strategies

Figure 7 shows the effect of cache size on the system latency of the three strategies, where the number of users is set to 3. From the results in Fig. 7, we can see that the latency of the three strategies decreases with the increased cache size. This is because that when the cache space becomes larger, there are more files to be cached in the server, which results in a smaller number of files being offloaded and computed. Moreover, the latency of the proposed strategy is smaller than that of PSO-random cache and PSO-FIFO cache when cache size is larger than 1, as the accuracy of caching files of the proposed scheme is better than that of the other two strategies due to the cache replacement strategy by user's recommendation. In contrast, PSO-random cache can not accurately cache the files with high popularity because of the random selection and random replacement, which results in higher

system latency. Similarly, because of the FIFO replacement, FIFO-random cache can not accurately cache the popular files and results in a large latency.

Figure 8 depicts the system latency of the three strategies versus the number of users, where the cache size is set to 5. From the results in Fig. 8, we can see that the latency of the three strategies increases with the number of users, since more files should be offloaded and computed, which results in a larger system latency. Moreover, the latency of PSO-random cache and PSO-FIFO cache is higher than that of the proposed strategy. This is because that the caching accuracy of the proposed strategy is better than that of the other two strategies. In contrast, PSO-random cache and PSO-FIFO cache consume much more latency due to the random replacement and FIFO replacement.

Figure 9 illustrates the system combination cost of the three strategies varies with the time slots, where the cache size is set to 5 and the number of users is set to 3. From the results in Fig. 9, we can observe that the combination cost of the proposed strategy becomes smaller with the time slot, and it becomes convergent after six time slots. Moreover, the effect of the proposed strategy on reducing combination cost is better than that of the PSO-without and PSO-random cache. This is because that the proposed strategy can effectively exploit the cache resources by caching the files according to the user's recommendation. In particular, the files with high popularity would not be replaced by the files with low popularity, so that the server can choose the top five files with the highest popularity. In contrast, the PSO-random cache decreases in the first six time slots, while it fluctuates from six time slots, due to the random replacement. The combination cost of the PSO-without cache keeps almost unchanged with the time slot, since it fails to use the cache resources.

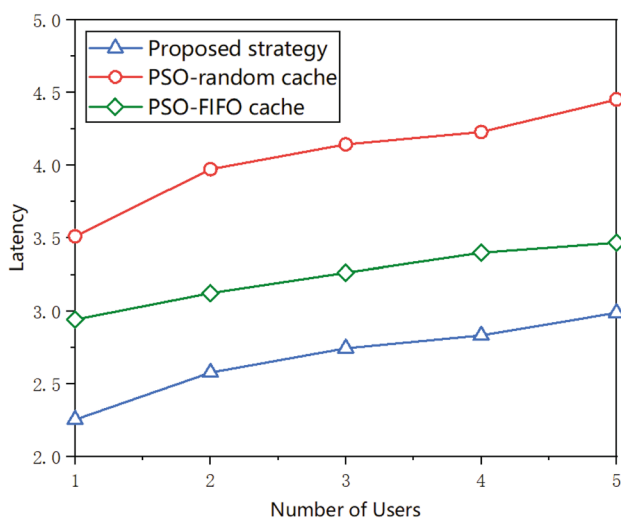


Fig. 8 Impact of the number of users on the latency of three strategies

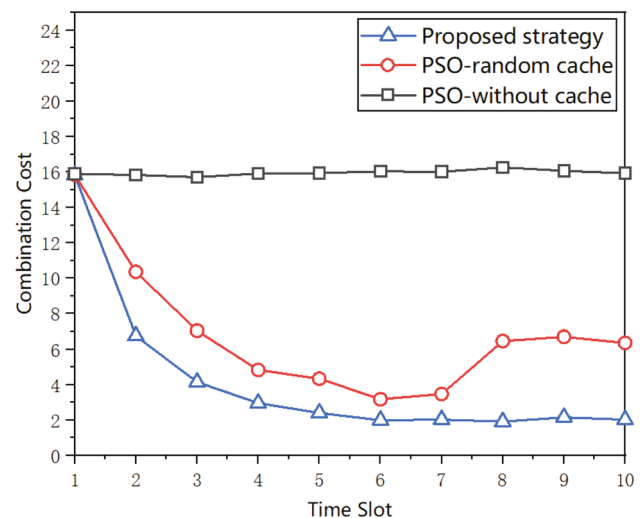


Fig. 9 Combination cost of three strategies versus time slot

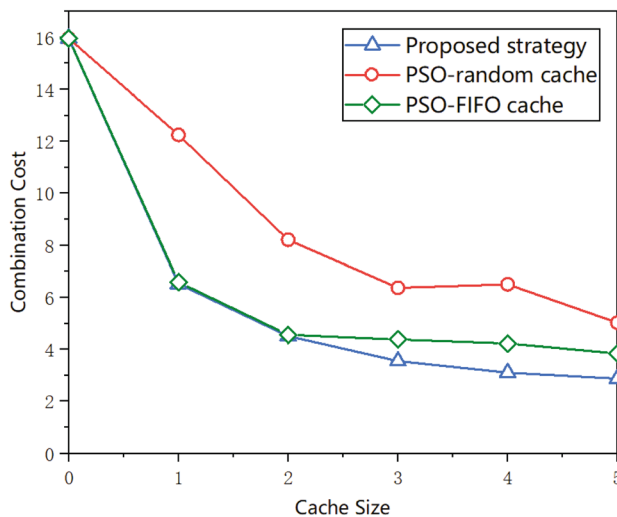


Fig. 10 Effect of cache size on the combination cost of three strategies

Figure 10 demonstrates the effect of cache size on the system combination cost of the three strategies, where the number of users is set to 3. From the results in Fig. 10, we can observe that the combination cost of the three strategies decreases with the increased cache size, as more files can be cached in the server, which results in a smaller number of files being offloaded and computed. Moreover, the combination cost of the proposed strategy is lower than that of the PSO-random cache and PSO-FIFO cache when cache size is larger than 1. This is because that the proposed strategy is based on the cache replacement strategy by the user's recommendation, and its accuracy of caching files is better than that of the other two strategies when cache size is large. In contrast, PSO-random cache can not accurately cache the files with high popularity due to the random selection and random replacement, which results in a larger system latency. Similarly, due to the FIFO replacement, FIFO-random cache can not accurately cache the popular files and results in a larger latency and energy.

Figure 11 shows the system combination cost of the three strategies versus the number of users, where the cache size is set to 5. From the results in Fig. 11, we can see that the combination cost of the three strategies increases with the number of users. The reason lies in that there are more files to be offloaded and computed when the number of users increases, which causes a larger system cost. Moreover, the combination cost of the proposed strategy is lower than that of PSO-random cache and PSO-FIFO cache, as its caching accuracy is better than that of the other two strategies. In contrast, PSO-random cache and PSO-FIFO cache consume much more latency and energy due to the random replacement and FIFO replacement.

From these phenomena in Figs. 3, 4, 5, 6, 7, 8, 9, 10, and 11, we can conclude that the proposed strategy can

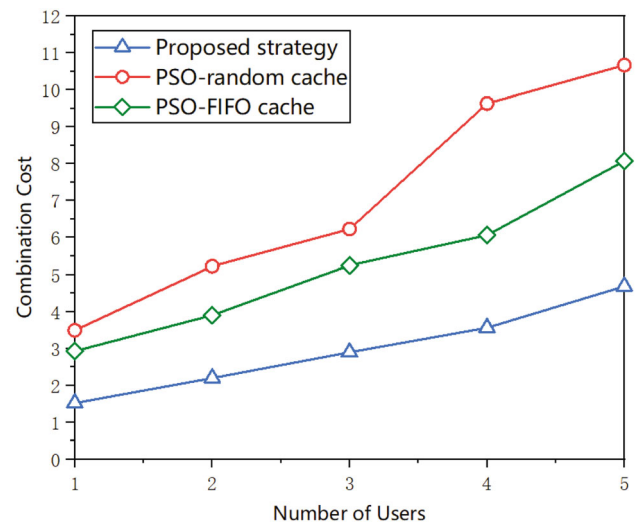


Fig. 11 Impact of the number of users on the combination cost of three strategies

outperform the two conventional strategies by effectively exploiting the caching resource and reduce the system combination cost.

5 Conclusion

This article proposed a PSO offloading strategy based on cache mechanism. In the computation offloading, PSO algorithm was used to optimize the offloading decision, which could reduce the latency and energy consumption of the system. Moreover, the cache mechanism was incorporated into the PSO offloading strategy to achieve a better effect of latency reducing and energy saving. The caching strategy proposed in this paper was a cache replacement strategy based on the user's recommendation. In each time slot, users requested popular files to the server for caching according to the popularity of the requested files, thus reducing the latency and energy consumption of computing in the next time slot. Simulation experiments were demonstrated to show that the latency and energy consumption could be effectively reduced by introducing cache strategy into the offloading strategy of edge computing. Compared with the random cache replacement strategy and FIFO cache replacement strategy, the proposed strategy could achieve a smaller latency and energy consumption. In future works, we will incorporate some other intelligent algorithms [38–41] into the proposed cache-enabled MEC networks, in order to reduce the system latency and energy consumption.

Acknowledgements This work was supported in part by the International Science and Technology Cooperation Projects of Guangdong Province (No. 2020A0505100060), Natural Science Foundation of

Guangdong Province (Nos. 2021A1515011392/2021A1515011812), and in part by the research program of Guangzhou University (No. YK2020008/YJ2021003).

Author contributions WZ designed the proposed offloading and caching strategy. LC helped conduct the simulation and verify the results. ST, LL and JX analyzed the application scenarios of the model, arranged the data, and proof read this paper. FZ was responsible for the check of the full paper and simulations. LF was responsible for checking the critical steps in the strategy design and improving the presentation style of the whole paper. All authors read and approved the final manuscript.

Funding This work was supported in part by the International Science and Technology Cooperation Projects of Guangdong Province (No. 2020A0505100060), Natural Science Foundation of Guangdong Province (Nos. 2021A1515011392/2021A1515011812), and in part by the research program of Guangzhou University (No. YK2020008/YJ2021003).

Data availability The authors state the data availability in this manuscript.

Code availability Not applicable.

Declarations

Conflict of interest The authors declare that there is no conflict of interest regarding the publication of this paper.

References

- Xie, H., Gao, F., Zhang, S., Jin, S.: A unified transmission strategy for TDD/FDD massive MIMO systems with spatial basis expansion model. *IEEE Trans. Veh. Technol.* **66**(4), 3170–3184 (2017)
- Hu, X., Zhong, C., Zhu, Y., Chen, X., Zhang, Z.: Programmable metasurface-based multicast systems: design and analysis. *IEEE J. Sel. Areas Commun.* **38**(8), 1763–1776 (2020)
- Kumar, B.A., Rao, P.T.: Optimized design and analysis approach of user detection by non cooperative detection computing methods in CR networks. *Clust. Comput.* **22**(4), 9777–9785 (2019)
- Zhao, J., Liu, J.: Future 5G-oriented system for urban rail transit: opportunities and challenges. *IEEE Trans. Veh. Technol.* **18**(2), 1–12 (2021)
- Lai, X.: Secure mobile edge computing networks in the presence of multiple eavesdroppers. *IEEE Trans. Commun.* 1–12 (2021)
- Zhao, J., Li, Q., Gong, Y., Zhang, K.: Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks. *IEEE Trans. Veh. Technol.* **68**(8), 7944–7956 (2019)
- Khan, S.I., Qadir, Z., Munawar, H.S., Nayak, S.R., Budati, A.K., Verma, K., Prakash, D.: UAVs path planning architecture for effective medical emergency response in future networks. *Phys. Commun.* **47**, 101337 (2021)
- Chen, L.: Intelligent ubiquitous computing for future UAV-enabled MEC network systems. *Clust. Comput.* **99**, 1–8 (2021)
- Lai, S.: Intelligent secure mobile edge computing for beyond 5G wireless networks. *Phys. Commun.* **45**(101283), 1–8 (2021)
- Guo, Y.: Efficient and flexible management for industrial internet of things: a federated learning approach. *Comput. Netw.* **192**, 1–9 (2021)
- Lai, S., Guo, Y.: Distributed machine learning for multiuser mobile edge computing systems. *IEEE J. Sel. Top. Signal Process.* **99**, 1–12 (2021)
- Li, C.: Dynamic offloading for multiuser multi-CAP MEC networks: a deep reinforcement learning approach. *IEEE Trans. Veh. Technol.* **70**(3), 2922–2927 (2021)
- Zhao, R.: Deep reinforcement learning based mobile edge computing for intelligent internet of things. *Phys. Commun.* **43**, 1–7 (2020)
- He, K.: Learning based signal detection for MIMO systems with unknown noise statistics. *IEEE Trans. Commun.* **69**, 3025–3038 (2021)
- He, K.: Ultra-reliable MU-MIMO detector based on deep learning for 5G/B5G-enabled IoT. *Phys. Commun.* **43**, 1–7 (2020)
- Tang, S.: Dilated convolution based CSI feedback compression for massive MIMO systems. *IEEE Trans. Veh. Technol.* **99**, 1–5 (2021)
- Xia, J., Fan, L.: Computational intelligence and deep reinforcement learning for next-generation industrial IoT. *IEEE Trans. Netw. Sci. Eng.* **99**, 1–12 (2021)
- Tang, S.: Battery-constrained federated edge learning in UAV-enabled IoT for B5G/6G networks. *Phys. Commun.* **47**(101381), 1–9 (2021)
- Zhao, Z.: System optimization of federated learning networks with a constrained latency. *IEEE Trans. Veh. Technol.* **99**, 1–5 (2021)
- Wang, X., Chen, M., Taleb, T., Ksentini, A., Leung, V.C.M.: Cache in the air: exploiting content caching and delivery techniques for 5G systems. *IEEE Commun. Mag.* **52**(2), 131–139 (2014)
- Xia, J.: Secure cache-aided multi-relay networks in the presence of multiple eavesdroppers. *IEEE Trans. Commun.* **67**(11), 7672–7685 (2019)
- Bi, S., Huang, L., Zhang, Y.A.: Joint optimization of service caching placement and computation offloading in mobile edge computing systems. *IEEE Trans. Wirel. Commun.* **19**(7), 4947–4963 (2020)
- Hu, X., Zhong, C., Zhang, Y., Chen, X., Zhang, Z.: Location information aided multiple intelligent reflecting surface systems. *IEEE Trans. Commun.* **68**(12), 7948–7962 (2020)
- He, L., He, K.: Towards optimally efficient search with deep learning for large-scale MIMO systems. *IEEE Trans. Commun.* (99), 1–12 (2022)
- Wang, Y., Adachi, F.: Multi-task learning for generalized automatic modulation classification under non-Gaussian noise with varying SNR conditions. *IEEE Trans. Wirel. Commun.* (2021). <https://doi.org/10.1109/TWC.2021.3052222>
- Hu, X., Wang, J., Zhong, C.: Statistical CSI based design for intelligent reflecting surface assisted MISO systems. *Sci. China: Inf. Sci.* **63**(12), 222303 (2020)
- Li, X., Mengyan, H., Liu, Y., Menon, V.G., Paul, A., Ding, Z.: IQ imbalance aware nonlinear wireless-powered relaying of B5G networks: security and reliability analysis. *IEEE Trans. Netw. Sci. Eng.* (2020). <https://doi.org/10.1109/TNSE.2020.3020950>
- Na, Z., Wang, J., Liu, C., Guan, M., Gao, Z.: Joint trajectory optimization and communication design for UAV-enabled OFDM networks. *Ad Hoc Netw.* **98**, 102031 (2020). <https://doi.org/10.1016/j.adhoc.2019.102031>
- Ale, L., Zhang, N., Fang, X., Chen, X., Wu, S., Li, L.: Delay-aware and energy-efficient computation offloading in mobile edge computing using deep reinforcement learning. *IEEE Trans. Cogn. Commun. Netw.* (2021). <https://doi.org/10.1109/TCCN.2021.3066619>
- Zhang, J., Hu, X., Ning, Z., Ngai, E.C.H., Zhou, L., Wei, J., Cheng, J., Hu, B.: Energy-latency tradeoff for energy-aware

- offloading in mobile edge computing networks. *IEEE Internet Things J.* **5**(4), 2633–2645 (2017)
31. Zhang, W., Wen, Y., Guan, K., Kilper, D., Luo, H., Wu, D.O.: Energy-optimal mobile cloud computing under stochastic wireless channel. *IEEE Trans. Wirel. Commun.* **12**(9), 4569–4581 (2013)
 32. Zhao, Z., Zhao, R., Xia, J., Lei, X., Li, D., Yuen, C., Fan, L.: A novel framework of three-hierarchical offloading optimization for MEC in industrial IoT networks. *IEEE Trans. Ind. Inform.* **16**(8), 5424–5434 (2020). <https://doi.org/10.1109/TII.2019.2949348>
 33. Yadav, R., Zhang, W., Kaiwartya, O., Song, H., Yu, S.: Energy-latency tradeoff for dynamic computation offloading in vehicular fog computing. *IEEE Trans. Veh. Technol.* **69**(12), 14198–14211 (2020). <https://doi.org/10.1109/TVT.2020.3040596>
 34. Chen, B., Yang, C.: Caching policy for cache-enabled D2D communications by learning user preference. *IEEE Trans. Commun.* **66**(12), 6586–6601 (2018)
 35. Cheng, C., Guo, L.: Machine learning-aided trajectory prediction and conflict detection for internet of aerial vehicles. *IEEE Internet Things J.* (2021). <https://doi.org/10.1109/JIOT.2021.3060904>
 36. Li, X., Wang, Q., Liu, M., Li, J., Peng, H., Piran, M.J., Li, L.: Cooperative wireless-powered noma relaying for B5G IoT networks with hardware impairments and channel estimation errors. *IEEE Internet Things J.* **8**(7), 5453–5467 (2021). <https://doi.org/10.1109/JIOT.2020.3029754>
 37. Na, Z., Liu, Y., Shi, J., Liu, C., Gao, Z.: UAV-supported clustered NOMA for 6G-enabled internet of things: trajectory planning and resource allocation. *IEEE Internet Things J.* (2020). <https://doi.org/10.1109/JIOT.2020.3004432>
 38. Wang, Z.: An adaptive deep learning-based UAV receiver design for coded MIMO with correlated noise. *Phys. Commun.* **45**(101295), 1–8 (2021)
 39. Li, X., Zhao, M., Zeng, M., Mumtaz, S., Menon, V.G., Ding, Z., Dobre, O.A.: Hardware impaired ambient backscatter NOMA systems: reliability and security. *IEEE Trans. Commun.* **69**(4), 2723–2736 (2021). <https://doi.org/10.1109/TCOMM.2021.3050503>
 40. Xia, J., Deng, D., Fan, D.: A note on implementation methodologies of deep learning-based signal detection for conventional MIMO transmitters. *IEEE Trans. Broadcast.* **66**(3), 744–745 (2020)
 41. Yang, J., Ruan, D., Huang, J., Kang, X., Shi, Y.: An embedding cost learning framework using GAN. *IEEE Trans. Inf. Forensics Secur.* **15**, 839–851 (2020)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Wenqi Zhou received the bachelor's degree in electromagnetic field and wireless technology from Nanjing University of posts and telecommunications, in 2020. She is currently pursuing the master's degree with the School of Computer Science and Cyber Engineering, Guangzhou University. Her current research interests focus on mobile edge computing, wireless caching and IoT networks.



Lunyuan Chen received the bachelor's degree in Communication Engineering from Xidian University in 2019. He is currently pursuing the master's degree with the school of Electronics and Communication Engineering, Guangzhou University. His current research interests focus on mobile edge computing, statistical machine learning and deep learning.



Shunpu Tang received the B.E. degree in 2020 and he is currently pursuing the master degree, both with the School of Computer Science and Cyber Engineering, Guangzhou University, Guangzhou, China. His current research interests include deep learning, distributed machine learning, IoT networks, mobile edge computing, machine learning in the wireless networks and deep reinforcement learning.



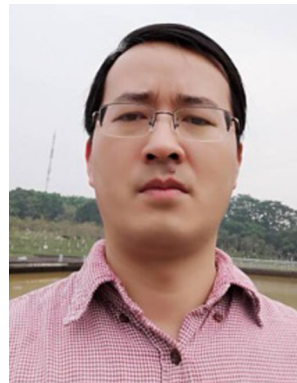
Lijia Lai received the bachelor's degree in School of Mechanical, Electrical & Information Engineering, Shandong University, WeiHai in 2014. He is currently pursuing the master's degree with the School of Computer Science and Cyber Engineering, Guangzhou University. His current research interests focus on federated learning and mobile edge computing.



Junjuan Xia received the bachelor degree from the Department of Computer Science, Tianjin University in 2003, and obtained the master degree from the Department of Electronic Engineering, Shantou University in 2015. Now she is working for the School of Computer Science, Guangzhou University. Her current research interests include wireless caching, IoT networks, physical-layer security, and cooperative relaying.



Fasheng Zhou received the Ph.D. degree in information and communication engineering from the South China University of Technology, Guangzhou, China. He received the M. Eng. and B. Eng. degrees both from the University of Science and Technology of China, Hefei, China. Since 2016, Dr. Zhou is with Guangzhou University, Guangzhou. From 2019 to 2020, he was a visiting scholar with the Communications Lab at National University of Singapore. His research interests include mobile edge computing and wireless signal processing.



Liseng Fan received the Ph.D. degree from the Tokyo Institute of Technology, Tokyo, in 2008. He is currently a Professor with the School of Computer Science and Cyber Engineering, Guangzhou University. He has published more than 40 articles on the IEEE journal and IEEE conferences. His main research interests include the information security, wireless networks, and the artificial intelligence. His recent research interest is the application of artificial intelligence into the wireless networks.